

## Caching with DAX – Part 2

### To Begin with the Lab

#### Summary of the Lab

A Node.js project was created to access **Amazon DynamoDB** through **Amazon DAX** using both the low-level DynamoDB API and the high-level Document Client. The Lambda application was packaged into a ZIP file, optimized by removing the AWS SDK, and deployed to **AWS Lambda**. An IAM role with permissions for **Amazon VPC**, **Amazon DAX**, **Amazon DynamoDB**, and **Amazon CloudWatch** was configured, and the Lambda function was connected to the same VPC as the DAX cluster. Finally, both handlers were tested successfully, verifying data retrieval in DynamoDB JSON format with the low-level API and plain JSON format with the Document Client.

---

#### Understanding Amazon DAX with AWS Lambda

**Amazon DAX (DynamoDB Accelerator)** is an in-memory cache that sits in front of **Amazon DynamoDB** to improve the performance of read-intensive applications. It uses the same DynamoDB API, allowing applications to benefit from caching with only minor code changes. Developers can access DAX using either the low-level DynamoDB API or the high-level Document Client depending on the required response format. This reduces database read latency and improves application responsiveness.

#### Example:

An application stores user activity in a **UserActivity** table with **UserId** as the partition key and **Timestamp** as the sort key. When users repeatedly request the same records, **Amazon DAX** serves the cached data from memory instead of querying **Amazon DynamoDB**, resulting in much faster response times.

---

#### Lab Steps

##### Step 1: Create a New Node.js Project

- Create a new folder for the project (for example, **Dax\_**).
- Open the folder in the terminal.
- Initialize a new Node.js project using **npm init**.
- Accept the default configuration.

```

PS C:\> cd Dax
PS C:\Dax_> npm init -y
Wrote to C:\Dax_\package.json:

{
  "name": "dax_",
  "version": "1.0.0",
  "description": "",
  "main": "index.js",
  "scripts": {
    "test": "echo \"Error: no test specified\" && exit 1"
  },
  "keywords": [],
  "author": "",
  "license": "ISC",
  "type": "commonjs"
}

PS C:\Dax_> dir

Directory: C:\Dax_

Mode                LastWriteTime         Length Name
----                -
-a-----          08-07-2026         13:28         240 package.json

```

## Step 2: Install the Amazon DAX Client

- Install the **Amazon DAX Client** package using NPM.
- Save the dependency in the project.
- **npm install amazon-dax-client --save**

```

PS C:\Dax_> npm install amazon-dax-client aws-sdk
npm warn deprecated querystring@0.2.0: The querystring API is considered Legacy. new code should use the URLSearchParams API instead.
npm warn deprecated uuid@3.4.0: uuid@10 and below is no longer supported. For ESM codebases, update to uuid@latest. For CommonJS codebases, use uuid@11 (but be aware this version will likely be deprecated in 2028).
npm warn deprecated uuid@8.0.0: uuid@10 and below is no longer supported. For ESM codebases, update to uuid@latest. For CommonJS codebases, use uuid@11 (but be aware this version will likely be deprecated in 2028).
npm warn deprecated aws-sdk@2.1693.0: The AWS SDK for JavaScript (v2) has reached end-of-support, and no longer receives updates. Please migrate your code to use AWS SDK for JavaScript (v3). More info https://a.co/cUPnyil

added 50 packages, and audited 51 packages in 6s

19 packages are looking for funding
  run `npm fund` for details

3 moderate severity vulnerabilities

Some issues need review, and may require choosing
a different dependency.

Run `npm audit` for details.
npm warn allow-scripts 1 package has install scripts not yet covered by allowScripts:
npm warn allow-scripts aws-sdk@2.1693.0 (postinstall: node scripts/warn-maintenance-mode.js)
npm warn allow-scripts
npm warn allow-scripts Run `npm approve-scripts --allow-scripts-pending` to review, or `npm approve-scripts <pkg>` to allow.
PS C:\Dax_> dir

Directory: C:\Dax_

Mode                LastWriteTime         Length Name
----                -
d-----          08-07-2026         13:37         node_modules
-a-----          08-07-2026        23316 package-lock.json
-a-----          08-07-2026         13:37         328 package.json

```

## Step 3: Create the Low-Level DynamoDB Lambda Handler

- Create a file named **DynamoDB.js**.
- Import the **AWS SDK** and **Amazon DAX Client**.

## Step 4: Package the Lambda Project

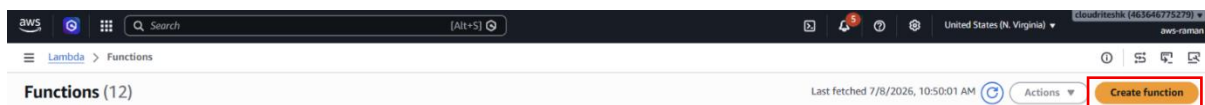
- Select all project files.
- Create a ZIP archive.
- Open the ZIP file.
- Navigate to the **node\_modules** folder.

- Delete the **aws-sdk** folder because the AWS SDK is already available in the **AWS Lambda** runtime.
- Save the ZIP file.

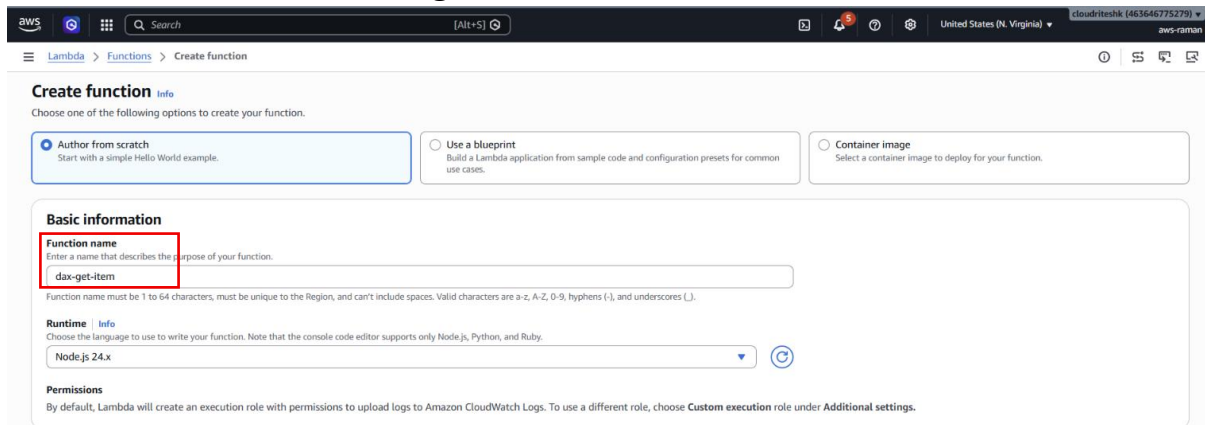
|              |                  |                    |           |
|--------------|------------------|--------------------|-----------|
| node_modules | 08-07-2026 13:37 | File folder        |           |
| dax_         | 08-07-2026 14:04 | Compressed (zip... | 14,513 KB |
| dynamodbjs   | 08-07-2026 13:54 | JSFile             | 0 KB      |
| package      | 08-07-2026 13:37 | JSON Source File   | 1 KB      |
| package-lock | 08-07-2026 13:37 | JSON Source File   | 23 KB     |

## Step 5: Create the Lambda Function

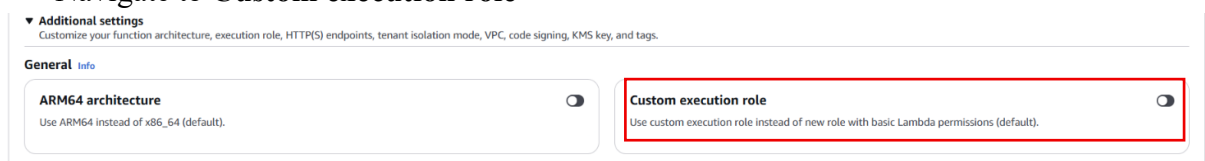
- Sign in to the **AWS Management Console**.
- Navigate to **AWS Lambda**.
- Click **Create function**.



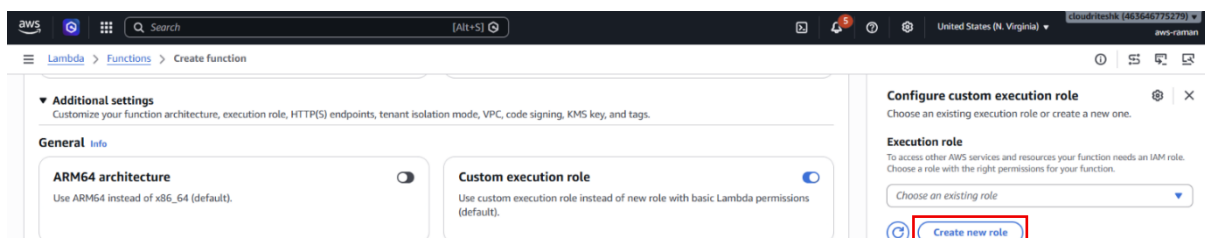
- Enter the function name **dax-get-item**.



- Go to **Additional setting**.
- Navigate to **Custom execution role**



- Click on the **Create new role**



- Create a new execution role named **dax-get-iteme-role-**

Define the role and role permissions you want to create for your Lambda function. For fully custom role creation [create role in IAM](#). 

### Role details

#### Role name

dax-get-item-role-

Maximum 64 characters. Use alphanumeric and +,=, @, \_ characters.

#### Use case

This role will allow your Lambda function to call AWS services on your behalf. [View trust policy](#).

- Open the IAM policy document.
- Click **Edit**.
- Paste the IAM policy JSON that grants access to **Amazon VPC**, **Amazon DAX**,
- **Amazon DynamoDB**, and **Amazon CloudWatch**.

### Edit role

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Sid": "CloudWatchLogs",  
6       "Effect": "Allow",  
7       "Action": [  
8         "logs:CreateLogGroup",  
9         "logs:CreateLogStream",  
10        "logs:PutLogEvents"  
11      ],  
12      "Resource": "*"  
13    },  
14    {  
15      "Sid": "DynamoDBAccess",  
16      "Effect": "Allow",  
17      "Action": [  
18        "dynamodb:GetItem",  
19        "dynamodb:BatchGetItem",  
20        "dynamodb:Query",  
21        "dynamodb:Scan",  
22        "dynamodb:PutItem",  
23        "dynamodb:UpdateItem",  
24        "dynamodb:DeleteItem"  
25      ],  
26      "Resource": "*"  
27    },  
28    {  
29      "Sid": "DAXAccess",  
30      "Effect": "Allow",  
31      "Action": [  
32        "dax:GetItem",
```

- Click **Allow**.
- Create the Lambda function.

### Step 6: Upload the Lambda Code

- Under **Function code**, choose **Upload from** → **.zip file**.

dax-get-item

Function overview Info

Diagram Template

 **dax-get-item**

 Layers (0)

[+ Add trigger](#) [+ Add destination](#)

[Export to Infrastructure Composer](#) [Download](#)

Function ARN  
 arn:aws:lambda:us-east-1:463646775279:function:dax-get-item

Description  
-

Last modified  
59 minutes ago

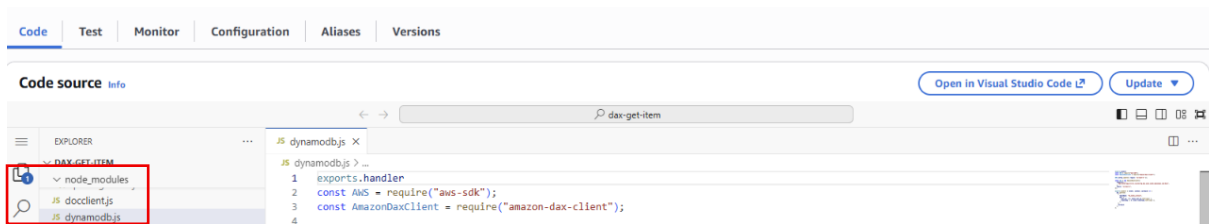
Code Test Monitor Configuration Aliases Versions

Code source Info

[Open in Visual Studio Code](#) [Update](#)

Update from a .zip file  
Update from a file in Amazon S3

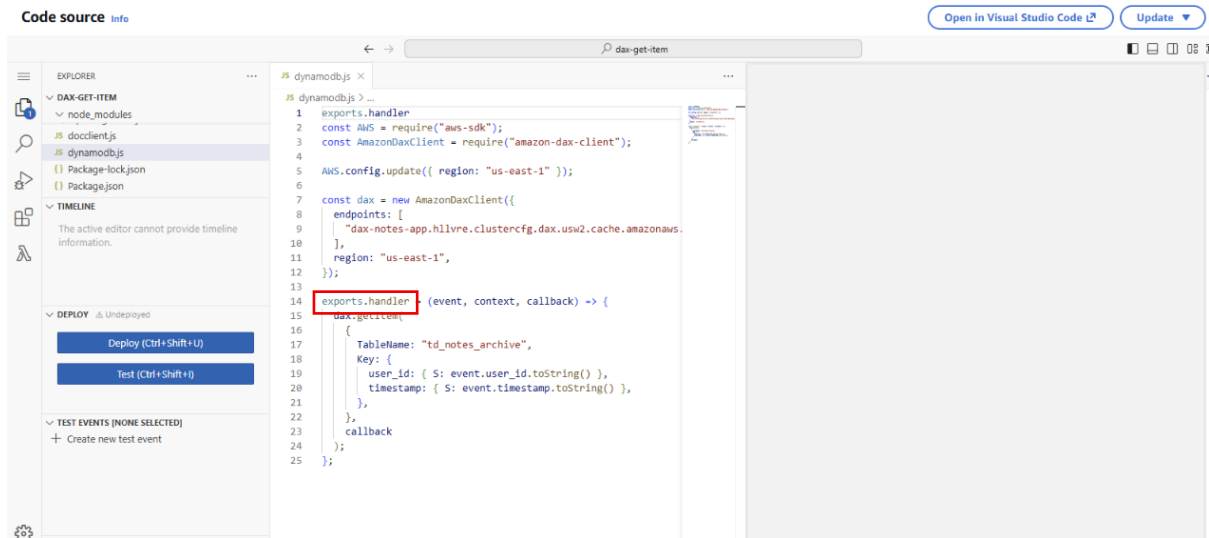
- Select the ZIP archive created earlier.
- ZIP file updated successfully



- If the upload fails, upload the ZIP file to **Amazon S3** and choose **Upload from Amazon S3**.

## Step 7: Configure the Low-Level DynamoDB Handler

- Set the handler name to: **DynamoDB.handler**



- Click on the Deploy

## Step 8: Configure the Lambda Network

- Go to Configuration → General configuration.
- Increase the Lambda timeout to **10 seconds**.

Edit basic settings

Basic settings

Description - optional

Memory

128 MB

Set memory to between 128 MB and 10240 MB

Ephemeral storage

512 MB

Set ephemeral storage (/tmp) to between 512 MB and 10240 MB

SnapStart

None

Supported runtimes: .NET 10 (C#/.NET Core), .NET 8 (C#/.NET Core), Java 11, Java 17, Java 21, Java 25, Python 3.12, Python 3.13, Python 3.14

Timeout

0 min 10 sec

Execution role

dax-get-item-role

Create new role

Edit role

View role details in IAM

Cancel Save

- Open the **Network** settings.
- Select the same **Amazon VPC** used by the DAX cluster.
- Select the required private subnets.
- Choose the corresponding security group.

- Save the changes.

#### Edit VPC

**VPC**

When you connect a function to a VPC in your account, it does not have access to the internet unless your VPC provides access. To give your function access to the internet, route outbound traffic to a NAT gateway in a public subnet. [Learn more](#)

**VPC** | Info  
Select a VPC for your resource to access.  
vpc-099792461c7d0a616 (172.31.0.0/16)

☐ Allow IPv6 traffic for dual-stack subnets  
You can allow outbound IPv6 traffic to subnets that have both IPv4 and IPv6 CIDR blocks.

**Subnets**  
Select the VPC subnets for Lambda to use to set up your VPC configuration.  
Choose subnets  
subnet-0d10d67b324b6c997 (172.31.32.0/20) us-east-1b subnet-0b5783ef03796c671 (172.31.16.0/20) us-east-1a

**Security groups**  
Select security groups for your VPC configuration. The following table shows the inbound and outbound rules for your selected security groups.  
Choose security groups  
sg-0032f8736b608a77d (launch-wizard-7)  
launch-wizard-7 created 2026-07-03T09:45:09.076Z

[View security group rules](#)

**Security group rules (2)** Last fetched 10 seconds ago

Inbound rules

| Security group ID    | Protocol   | Ports | Source    |
|----------------------|------------|-------|-----------|
| sg-0032f8736b608a77d | Custom TCP | 80    | 0.0.0.0/0 |
| sg-0032f8736b608a77d | Custom TCP | 22    | 0.0.0.0/0 |

### Step 9: Test the Low-Level DynamoDB Handler

- Click **Test**.
- Create a new test event named **testevent**.
- Provide the following event:

```
{
  "user_id": "A",
  "timestamp": "1"
}
```

- Save the test event.

**Code source** Info

[Open in Visual Studio Code](#) [Update](#)

EXPLORE | DAX-GET-ITEM | node\_modules | docclient.js | dynamodb.js | Package-lock.json | Package.json | TIMELINE | DEPLOY | TEST EVENTS

```
1 exports.handler = async function(event, context, callback) {
2   const AWS = require("aws-sdk");
3   const AmazonDAXClient = require("amazon-dax-client");
4
5   AWS.config.update({ region: "us-east-1" });
6
7   const dax = new AmazonDAXClient({
8     endpoints: [
9       "dax-notes-app.hllvre.clustercfg.dax.usw2.cache.amazonaws.com"
10    ],
11    region: "us-east-1",
12  });
13
14  exports.handler = (event, context, callback) => {
15    dax.getItem(
16      {
17        TableName: "td_notes_archive",
18        Key: {
19          user_id: { S: event.user_id.toString() },
20          timestamp: { S: event.timestamp.toString() },
21        },
22      },
23      callback
24    );
25  };
26};
```

**Create new test event**

Event Name  
testevent

Maximum of 25 characters consisting of letters, numbers, dots, hyphens and underscores.

Invocation type

☒ Synchronous  
Executes the Lambda function and blocks until receiving the function's response, with a maximum timeout of 15 minutes. Returns function output or error details directly to the calling application.

☐ Asynchronous  
Enqueues the Lambda function for execution and returns immediately with a request ID. Function processes independently, with results optionally sent to a configured destination like SQS, SNS, or EventBridge.

Event sharing settings

☒ Private  
This event is only available in the Lambda Console and to the event creator. You can configure a total of ten. [Learn more](#)

☐ Shareable  
This event is available to IAM users within the same account who have permissions to access and use shareable events. [Learn more](#)

Template - optional  
Hello World

Event JSON

```
1 {
2   "user_id": "A",
3   "timestamp": "1"
4 }
5
```

- Run the test.
- Verify that the item is returned successfully in **DynamoDB JSON** format.

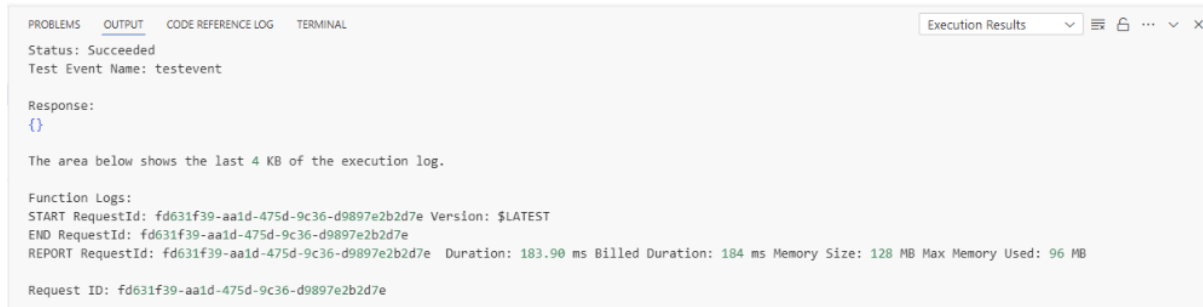
### Step 10: Test the Document Client Handler

- Change the Lambda handler to:



```
JS indexjs > ...
1 const AWS = require("aws-sdk");
2 const AmazonDaxClient = require("amazon-dax-client");
3 AWS.config.update({ region: "us-east-1" });
4 const dax = new AmazonDaxClient({
5   endpoints: [
6     "dax://dax-notes-app.vr10te.dax-clusters.us-east-1.amazonaws.com",
7   ],
8   region: "us-east-1",
9 });
10 Lambda.handler = (event, context, callback) => { dax.getItem(
11   { {} Lambda
12     TableName: "td_notes_archive",
13     Key: {
14       user_id: { S: event.user_id.toString() },
15       timestamp: { S: event.timestamp.toString() },
16     },
17   },
18   callback
19 );
20 };
```

- Save the function.
- Run the same test event again.
- Verify that the response is returned successfully in standard JSON format.
- Confirm that both the low-level API and the **Amazon DynamoDB Document Client** retrieve data through **Amazon DAX** successfully.



PROBLEMS OUTPUT CODE REFERENCE LOG TERMINAL Execution Results

Status: Succeeded  
Test Event Name: testevent

Response:  
{}

The area below shows the last 4 KB of the execution log.

Function Logs:  
START RequestId: fd631f39-aa1d-475d-9c36-d9897e2b2d7e Version: \$LATEST  
END RequestId: fd631f39-aa1d-475d-9c36-d9897e2b2d7e  
REPORT RequestId: fd631f39-aa1d-475d-9c36-d9897e2b2d7e Duration: 183.90 ms Billed Duration: 184 ms Memory Size: 128 MB Max Memory Used: 96 MB

Request ID: fd631f39-aa1d-475d-9c36-d9897e2b2d7e